



Java et la Programmation Orientée Objet



Houssem MAHMOUDI
houssem.mahmoudi@ensicaen.fr

Ensicaen©2025 - HM



Chapitre 5 : Les Threads

1. Introduction
2. Notion d'un thread
3. Création d'un thread en Java
 - a Classe Thread
 - b Interface Runnable
4. Cycle de vie d'un Thread en Java
5. La synchronisation
6. Communication inter-Thread
7. Applications

Ensicaen©2025 - HM

2

I. Introduction

- Un **processus** est un programme en cours d'exécution.
- **Multitâche** : la capacité d'un processeur à exécuter plusieurs programmes simultanés.
- Un programme en Java s'exécute à partir de la méthode main() d'une façon séquentielle (une instruction après l'autre).

Exemple :

🔗 Analysez le code Java et déduisez le résultat final

- En appelant la méthode **chatServeur()**, le programme entre dans une boucle infinie !!
- Pas de chance d'exécuter la méthode **downloadFile()** !!



Comment un programme peut-il exécuter plusieurs tâches en parallèle ?

II. Notion d'un Thread

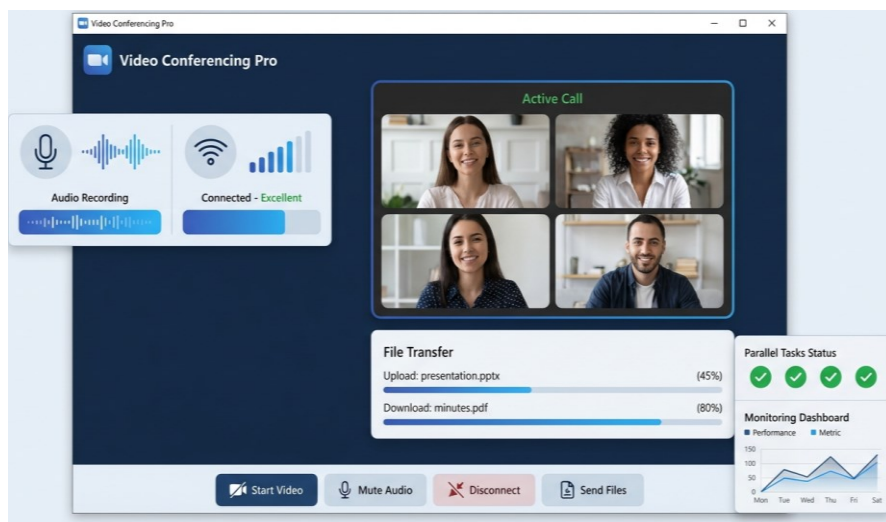
- En Java, pour gérer l'exécution simultanée de plusieurs tâches, un programme peut être divisé en plusieurs **threads**.
- Un **thread** est une partie du code qui exécute une tâche indépendante (séquence d'instructions) en parallèle avec le reste du programme.
- Un programme Java peut contenir plusieurs threads qui s'exécute simultanément, c'est ce qu'on appelle le **Multithreading**.
- Le **Multithreading** est une technique qui permet à un programme d'exécuter plusieurs threads en parallèle, partageant les mêmes ressources du processus principal (programme Java qui a crée le Thread).
- Un programme Java démarre avec un thread principal appelé **main Thread**, qui exécute la méthode **main(String[] args)**.

II. Notion d'un Thread

- **Pourquoi utiliser le multithreading ?**
 - Améliorer les performances : Exploitant la puissance du processeur.
 - Optimiser l'utilisation des ressources : Eviter les temps d'attente inutiles.
 - Rendre les applications plus réactives.
- **Exemple : Application Visioconférence**
 - Plusieurs tâches qui s'exécutent en parallèle :
 - Afficher la vidéo en temps réel.
 - Sauvegarder l'audio à partir du microphone.
 - Gérer la connexion réseau.
 - Envoi / réception des fichiers.

II. Notion d'un Thread

- Application **Visioconférence**



III. Création d'un Thread

- Par défaut, un programme Java s'exécute avec un seul thread : c'est le **thread principal** qui exécute la méthode `main()`.
- Pour exécuter plusieurs tâches en parallèle, on doit créer des threads secondaires.
- Il existe deux façons de créer un thread :
 - Hériter de la classe prédéfinie **Thread**.
 - Implémenter l'interface **Runnable**.

1. Classe Thread

- La classe **Thread** fait partie du package `java.lang` permet de créer et gérer des threads.
- Créez une classe qui hérite de la classe **Thread**.
- Redéfinit la méthode **run()** : qui contient le code à exécuter par le thread.
- Dans la méthode `main()`, créez un objet de votre classe.
- Invoquer la méthode **start()** ➔ ce qui permet de lancer un nouveau thread.

III. Création d'un Thread – Exemple : Thread

```
class MyServer2 extends Thread {
    public void run() {
        setName("Chat_with_Server");
        int i=1;
        while(true) {
            System.out.println(i + " Chat with server"); i++;
            Thread.sleep(500);
        }
    }
}
class MyDownload2 extends Thread {
    public void run() {
        setName("Downloading_Files");
        int i=1;
        while(true) {
            System.out.println(i + " Downloading files.."); i++;
        }
    }
}
public class IntroThreadSol {
    public static void main(String[] args) {
        //Chatting with server
        MyServer2 mys = new MyServer2();
        mys.start();
        // Downloading file
        MyDownload2 myd = new MyDownload2();
        myd.start();
    }
}
```

III. Création d'un Thread

2. Interface Runnable

- L'interface **Runnable** fait partie du package `java.lang` et permet de définir une tâche qui sera exécutée par un thread.
- Créez une classe qui implémente de l'interface `Runnable`.
- Redéfinit la méthode **run()** (La seule méthode qui existe dans l'interface), ce code sera exécuter par le thread.
- Dans la méthode `main()`, créez un objet de votre classe, puis créez un objet de la classe Thread en lui passant l'objet de la classe comme paramètre du constructeur.
- Invoquer la méthode `start()` : ce qui permet de lancer un nouveau thread.

↳ L'utilisation de l'interface `Runnable` est préférable si la classe doit hériter d'une autre classe.

III. Création d'un Thread – Exemple : Runnable

```
class MyServer3 implements Runnable{
    public void run() {
        int i=1;
        while(true) {
            System.out.println(i + " Chat with server"); i++;
        }
    }
}
class MyDownload3 implements Runnable {
    public void run() {
        int i=1;
        while(true) {
            System.out.println(i + " Downloading files.."); i++;
        }
    }
}
public class IntroThreadSolRunnable {
    public static void main(String[] args) {
        //Chatting with server
        MyServer3 mys = new MyServer3();
        Thread t1 = new Thread(mys, "Chat_With_Server");
        t1.start();
        // Downloading file
        MyDownload3 myd = new MyDownload3();
        Thread t2 = new Thread(myd);
        t2.setName("Downloading_Files");
        t2.start();
    }
}
```

III. Création d'un Thread – Observation des threads

```

"Chat_With_Server" #15 prio=5 os_prio=0 cpu=24109.38ms elapsed=32.06s tid=0x000001cd8c321010 nid=0x17c0 runnable [0x0000006f16dfe0
java.lang.Thread.State: RUNNABLE
    at java.io.FileOutputStream.writeBytes(java.base@17.0.10/Native Method)
    at java.io.FileOutputStream.write(java.base@17.0.10/FileOutputStream.java:349)
    at java.io.BufferedOutputStream.flushBuffer(java.base@17.0.10/BufferedOutputStream.java:81)
    at java.io.BufferedOutputStream.flush(java.base@17.0.10/BufferedOutputStream.java:142)
    - locked <0x0000000707c0b390> (a java.io.BufferedOutputStream)
    at java.io.PrintStream.write(java.base@17.0.10/PrintStream.java:570)
    - locked <0x0000000707c10b40> (a java.io.PrintStream)
    at sun.nio.cs.StreamEncoder.writeBytes(java.base@17.0.10/StreamEncoder.java:234)
    at sun.nio.cs.StreamEncoder.implFlushBuffer(java.base@17.0.10/StreamEncoder.java:313)
    at sun.nio.cs.StreamEncoder.flushBuffer(java.base@17.0.10/StreamEncoder.java:111)
    - locked <0x0000000707c0b378> (a java.io.OutputStreamWriter)
    at java.io.OutputStreamWriter.flushBuffer(java.base@17.0.10/OutputStreamWriter.java:178)
    at java.io.PrintStream.writeln(java.base@17.0.10/PrintStream.java:723)
    - locked <0x0000000707c10b40> (a java.io.PrintStream)
    at java.io.PrintStream.println(java.base@17.0.10/PrintStream.java:1028)
    at intro.MyServer2.run(Chapitre5/IntroThreadSol.java:8)

'Downloading_Files' #16 prio=5 os_prio=0 cpu=24796.88ms elapsed=32.05s tid=0x000001cd8c9f8810 nid=0x147c waiting for monitor entry
000j
java.lang.Thread.State: BLOCKED (on object monitor)
    at java.io.PrintStream.writeln(java.base@17.0.10/PrintStream.java:718)
    - waiting to lock <0x0000000707c10b40> (a java.io.PrintStream)
    at java.io.PrintStream.println(java.base@17.0.10/PrintStream.java:1028)
    at intro.MyDownload2.run(Chapitre5/IntroThreadSol.java:16)
  
```

III. Création d'un Thread

Constructeurs de la classe Thread :

- **Thread()** : Crée un thread sans nom ni tâche spécifique
 ↳ Thread d = new Thread();
- **Thread(String name)** : Crée un thread avec un nom ni tâche spécifique
 ↳ Thread d = new Thread("Download");
- **Thread(Runnable obj)** : Crée un thread qui exécute la méthode run() de l'objet qui implémente l'interface Runnable
 ↳ Thread t1 = new Thread(mys); //mys doit implémenter l'interface Runnable.
- **Thread(Runnable obj, String name)** : Crée un thread avec un nom spécifié
 ↳ Thread t1 = new Thread(mys, "Chat_With_Server");

III. Création d'un Thread

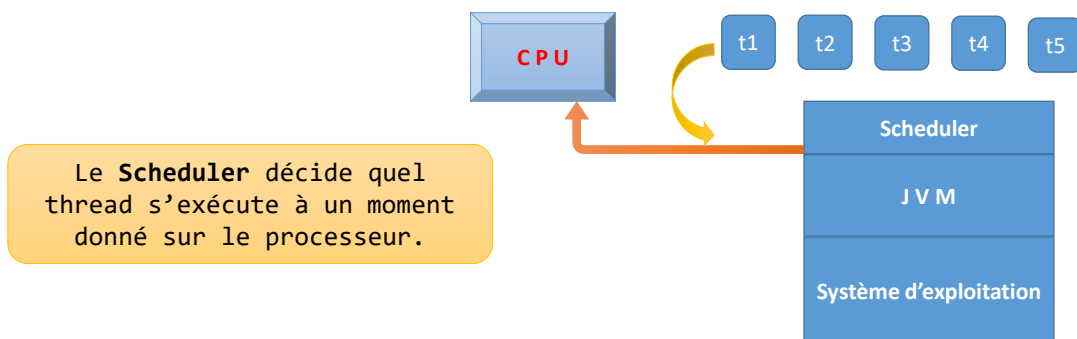
Méthodes principales :

- **start()** : Lance l'exécution du thread en appelant la méthode run()
- **run()** : Contient le code à exécuter par le thread. Elle ne doit pas être appelée directement.
- **join()** : Attendre la fin d'un thread avant de continuer l'exécution.
- **interrupt()** : Interrompt le thread (lève une InterruptedException).
- **isInterrupted()** : Retourne true si le thread a été interrompu (par un appel à la méthode interrupt()), et false sinon.
- **isAlive()** : Retourne true si le thread est en cours d'exécution, false sinon.
- **getState()** : Retourne l'état du thread.
- **setName(String name)** : Définit le nom du thread.
- **setPriority(int priority)** : Définit la priorité du thread (entre *Thread.MIN_PRIORITY* = **1** et *Thread.MAX_PRIORITY* = **10**). La priorité normale du thread *Thread.NORM_PRIORITY* = **5**

III. Création d'un Thread

Méthodes statiques :

- **Thread.currentThread()** : Retourne une référence au thread en cours d'exécution.
- **Thread.sleep(long millis)** : Met le thread courant en pause en millisecondes.
- **Thread.yield()** : Suggère au planificateur de threads (**Scheduler**) de céder temporairement le temps d'exécution du thread actuel à autres threads.



IV. Cycle de vie d'un Thread en Java

- Un thread en Java traverse plusieurs **états** depuis sa création jusqu'à sa fin d'exécution.
- Ces états sont gérés par la JVM (Java Virtual Machine) et sont définis dans l'énumération *Thread.State*
- NEW** : Le thread est créé, mais il n'a pas encore démarré, objet créé mais la méthode *start()* pas encore appelée.

↳ Exemple :

```
Thread t = new Thread();
System.out.println(t.getState()); // Affiche : NEW
```

- Running** : Le thread est prêt à s'exécuter (attente de Scheduler) ou est en cours d'exécution.

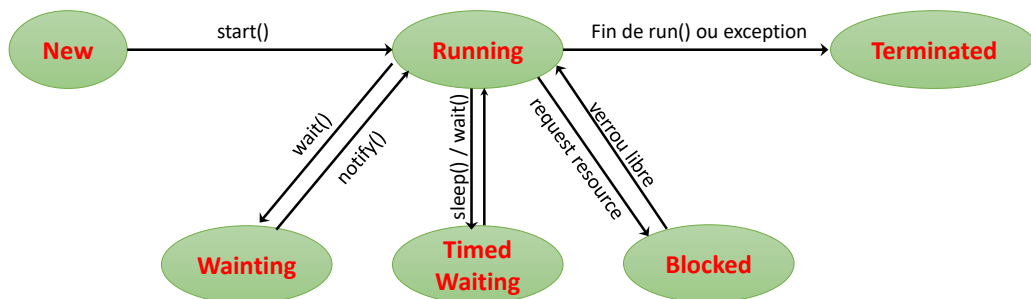
↳ Exemple :

```
Thread t = new Thread();
t.start();
System.out.println(t.getState()); // Peut afficher : RUNNABLE
```

- WAITING** : Le thread est en attente indéfinie d'un autre thread pour effectuer une action.

IV. Cycle de vie d'un Thread en Java

- BLOCKED** : Un thread passe dans cet état lorsqu'il veut accéder à une ressource protégée par un verrou (**lock**), mais que cette ressource est déjà détenue par un autre thread.
- TIMED_WAITING** : Similaire à WAITING, mais avec une durée limitée. Le thread attend pendant un temps spécifié via *sleep(ms)*, *wait(timeout)*, ou *join(timeout)*.
- TERMINATED** : Le thread a terminé son exécution.



↳ Un **verrou** est un mécanisme de synchronisation qui garantit qu'un seul thread peut accéder à une ressource partagée. Cela se fait avec le mot-clé **synchronized**.

V. La synchronisation en Java

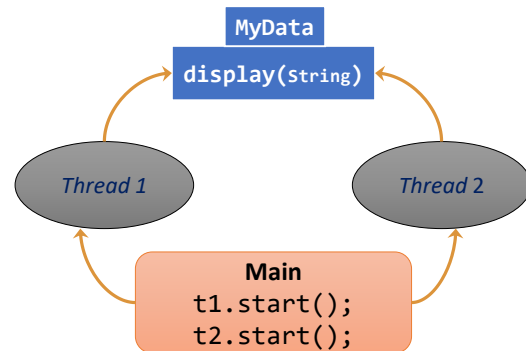
- La synchronisation est un mécanisme qui permet de contrôler l'accès de plusieurs threads à des ressources partagées.
- La synchronisation en Java repose sur le concept de **moniteur (verrou)**, qui est un mécanisme permettant d'empêcher plusieurs threads d'accéder simultanément à une méthode ou une ressource partagée.

Exemple :

Analyser l'exécution de ce programme :

↳ deux threads tente d'appeler au même temps la méthode `display()`

Problème : Affichage n'est pas synchrone !



V. La synchronisation en Java

- Le mot-clé **synchronized** garantit qu'une méthode ou un bloc de code ne peut être exécuté que par un seul thread à la fois, en **verrouillant son accès** aux autres threads.
- synchronized** garantit un accès mutuellement exclusif (mutex) à la ressource partagée.

Syntaxe :

- Pour synchroniser une méthode :

↳ `public synchronized void maMéthode() {.....}`

- Pour synchroniser un bloc de code :

↳ `synchronized(Objet) { ... .. }`

Exercice 1:

Modifiez l'exemple précédent afin d'afficher d'abord le texte du **thread1**, puis celui du **thread2**.

VI. Communication inter-Thread

- La communication inter-thread en Java est un mécanisme permettant à plusieurs threads de collaborer en échangeant des informations.
- Les threads communiquent en utilisant les méthodes `wait()`, `notify()`, et `notifyAll()`
- Ces méthodes doivent être utilisées dans des blocs ou méthodes synchronisées pour garantir un accès sécurisé aux ressources partagées.
 - **`wait()`** : Libérer le verrou et met le thread en attente (suspendre son exécution).
 - **`notify()`** : Réveille un thread en attente pour continuer son exécution.
 - **`notifyAll()`** : Réveille tous les threads qui sont en attente.

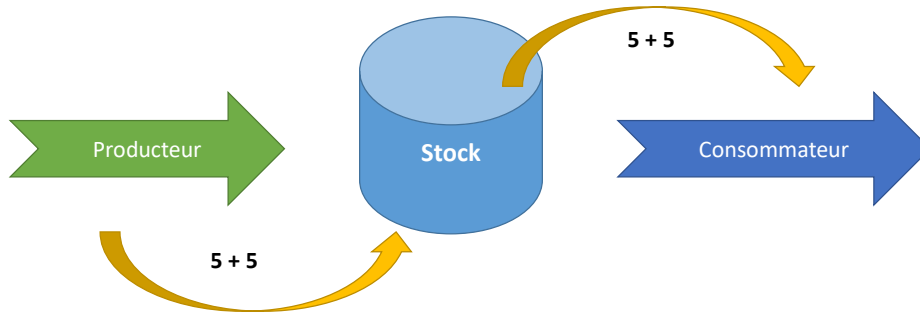
VI. Communication inter-Thread

Exercice 2:

- On souhaite simuler un système de gestion de stock où un producteur ajoute 10 produits et un consommateur les retire. Le stock est limité à 5 produits maximum. Le producteur ajoute d'abord 5 produits dans le stock, puis attend que le consommateur les retire avant de continuer.
- Implémenter la classe **Stock** contenant :
 - Une variable produit représentant le nombre de produits en stock.
 - Une méthode `produire()` permettant au producteur d'ajouter un produit si le stock n'est pas plein (≤ 5), sinon il doit attendre.
 - Une méthode `consommer()` permettant au consommateur de retirer un produit si le stock n'est pas vide (≥ 0), sinon il doit attendre.
- Implémenter les classes suivantes :
 - **Producteur**, qui hérite de `Thread` et exécute la méthode `produire()` 10 fois.
 - **Consommateur**, qui hérite aussi de `Thread` et exécute la méthode `consommer()` 10 fois.
 - **TestStock**, qui lance les deux threads pour l'affichage du stock.

VI. Communication inter-Thread

Exercice 2:



Si le **stock = 5**

- Il faut réveiller le thread consommateur
- Le producteur doit attendre

Si le **stock = 0**

- Il faut réveiller le thread producteur
- Le consommateur doit attendre

VI. Communication inter-Thread

Exercice 3: (Challenge)

- Implémenter un système de gestion de stock en utilisant la communication entre threads en Java. **n** est le nombre des séries (alimentations).
- Un **Chargeur** permet d'alimenter un tableau par trois entiers (<100) aléatoire.
- Après chaque série de 3 ajouts, le chargeur doit attendre que le consommateur retire ces 3 entiers les calcule et affiche leur somme.
- Le **Consommateur** doit retirer les 3 entiers un par un et réveiller le chargeur une fois qu'il a tous retirés.
- La synchronisation doit être gérée avec wait() et notify() pour éviter les accès concurrents incorrects.